

Week 3

Cloud Deployment, DevOps and Security Practices

From localhost to a secure deployed product

For M.Sc. Computer Science Students | 30 Hours

Agenda

This session sequence is designed for teaching and live coding.

1

Prepare code for production

2

Understand environment variables

3

Deploy frontend and backend

4

Connect cloud database

5

Fix CORS and configuration issues

6

Add basic testing and CI/CD

7

Apply OWASP security practices

8

Create deployment documentation

What you will learn after completing this week

By the end of this week, students should be able to:



Explain localhost vs production environment



Configure CORS and API base URLs



Use environment variables safely



Create basic GitHub Actions workflow



Deploy frontend application



Apply basic OWASP security checks



Deploy backend API service



Prepare deployment guide



Connect to a cloud database



Demo a live application URL

Goal of the Week: Ship the Product

A project becomes more valuable when it is live and usable.

- Move from local machine to internet-accessible deployment.
- Separate frontend, backend and database hosting.
- Use environment variables for secrets and URLs.
- Test API endpoints after deployment.
- Apply security and reliability improvements.
- Create a deployment guide so another developer can repeat the process.

Instructor cue: Ask: Is a project truly complete if nobody else can run or access it?

Localhost vs Production

Localhost

Runs only on student machine. Useful for development and debugging.

Production

Runs on hosted infrastructure and can be accessed by users.

Config

Development uses local URLs; production uses deployed URLs.

Secrets

Use environment variables; never commit passwords or API keys.

Reliability

App must handle errors, restarts, slow networks and invalid inputs.

Security

Authorization, validation, hashing and safe deployment settings matter.

Deployment Architecture

Teach the path of one request through the deployed system.

- User opens frontend URL hosted on Vercel/Netlify.
- Frontend sends request to backend URL hosted on Render/Railway.
- Backend validates request and talks to cloud database.
- Cloud database returns records to backend.
- Backend returns JSON response to frontend.
- Frontend renders updated user interface.

Instructor cue: Draw this on board and ask students to identify where each error can happen.

Environment Variables

- Environment variables separate code from configuration.
- Never put secrets directly in code or GitHub.
- Use .env locally and platform settings in production.
- Document required variable names without exposing values.

```
# .env example for backend
PORT=5000
MONGODB_URI=mongodb+srv://...
JWT_SECRET=replace_with_strong_secret
CLIENT_URL=https://your-frontend.vercel.app

# Use in Node.js
const port = process.env.PORT || 5000;
const dbUrl = process.env.MONGODB_URI;
```

Frontend Deployment Flow

Example using Vercel or Netlify.

- Push frontend code to GitHub.
- Import repository into deployment platform.
- Set build command such as `npm run build`.
- Set output directory such as `dist` or `build`.
- Add frontend environment variable for backend API URL.
- Deploy and test routes, refresh behavior and API calls.

Instructor cue: Frontend deployment is usually simple; wrong API URL is the most common issue.

Backend Deployment Flow

Example using Render or similar platform.

- Push backend code to GitHub.
- Create web service and connect repository.
- Set build command such as `npm install`.
- Set start command such as `npm start` or `node server.js`.
- Add environment variables in platform dashboard.
- Check server logs after first deploy.
- Test health route and API endpoints with Postman.

Instructor cue: Show logs live; students must learn that production debugging starts with logs.

Cloud Database Connection

A deployed backend needs a database it can reach online.

- Create database user with limited required access.
- Add network access rules carefully.
- Copy connection string and store it as environment variable.
- Test connection locally before deployment.
- Test create/read/update/delete after deployment.
- Avoid exposing database credentials in README screenshots.

Instructor cue: Discuss why credentials and screenshots can leak secrets.

CORS and API Configuration

- CORS controls which origins can call the backend.
- Use production frontend URL in backend CORS config.
- Keep frontend API base URL configurable.
- Test with browser Network panel and Postman.

```
const cors = require("cors");

app.use(cors({
  origin: process.env.CLIENT_URL,
  methods: ["GET", "POST", "PUT", "DELETE"],
  credentials: true
}));

// Frontend config
const API_URL = import.meta.env.VITE_API_URL;
```

Basic CI/CD with GitHub Actions

CI/CD automates checks whenever code changes.

- CI means continuous integration: run checks on code changes.
- CD means continuous delivery/deployment: ship changes safely.
- Start simple: install dependencies and run tests/lint.
- Use GitHub Actions workflow file in `.github/workflows`.
- Deployment platforms often auto-deploy on push to main.
- A failed build should stop unsafe changes from reaching users.

Instructor cue: Do not over-teach DevOps; focus on why teams automate repeated checks.

Simple GitHub Actions Workflow

- Run on push and pull request.
- Install dependencies.
- Run tests or lint command.
- Use as a quality gate for student projects.

```
name: Node CI

on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm install
      - run: npm test --if-present
```

Security Practices Using OWASP Mindset

Security must be introduced as part of development, not after it.

- Broken access control: verify roles on backend, not only in UI.
- Cryptographic failures: hash passwords and use HTTPS-hosted services.
- Injection: validate and sanitize inputs; avoid unsafe queries.
- Security misconfiguration: do not expose secrets or debug errors.
- Authentication failures: enforce strong password and session practices.
- Logging failures: keep enough logs to investigate issues.

Instructor cue: Map each OWASP risk to one project feature.

Testing After Deployment

Smoke Test

Open app, login and check primary workflow.

API Test

Use Postman against deployed backend URL.

Database Test

Verify records are created in cloud database.

Invalid Input

Test empty fields, invalid email and unauthorized access.

Responsive

Check app on mobile and laptop screen sizes.

Regression

Confirm Week 2 features still work after deployment.

Five-Day Implementation Plan

Day	Focus	Student Task	Evidence
1	Prepare	Config cleanup, .env, build and production readiness.	Code review checklist
2	Deploy	Deploy frontend and backend services.	Live URLs
3	Database	Connect cloud database and test CRUD.	Cloud DB screenshots
4	Quality	Add validation, logs and security fixes.	Test/security checklist
5	CI/CD	Add GitHub Actions, deployment guide and demo.	Guide and final live demo

Assessment and Weekly Deliverables

Live deployment proves students can ship, not only code.

- Live frontend URL and backend API URL.
- Cloud database connected successfully.
- Environment variable list without secret values.
- Deployment guide with steps and troubleshooting.
- Security checklist mapped to project features.
- Postman screenshots for deployed APIs.
- GitHub Actions workflow or documented deployment automation.
- Weekly progress report and demo.

Instructor cue: During review, ask students to redeploy one change live.

Study Materials and Reference Links

Use official documentation first. These links are suitable for student practice and instructor preparation.

[Vercel: Deploy React](#)

Deploy React frontend with Git integration.

[Docker Get Started](#)

Optional containerization concepts and practice.

[Render: Deploy Node Express App](#)

Deploy Express backend as a web service.

[Node Environment Variables](#)

Reading process environment values in Node.js.

[MongoDB Atlas: Connect to Cluster](#)

Cloud database connection and access setup.

[MDN CORS Guide](#)

Browser cross-origin request concept.

[GitHub Actions Docs](#)

CI/CD workflow automation from GitHub repositories.

[Jest Docs](#)

JavaScript testing framework starter docs.

[OWASP Top 10: 2021](#)

Awareness document for critical web security risks.

[Postman Learning Center](#)

API requests, collections, documentation and testing.

Tip: Assign the linked docs as pre-read before the lab and ask students to note 3 learnings and 2 doubts.

End of Week 3 Teaching Deck

Next action: run the lab, collect GitHub links, review commits, and record weekly progress.

Student evidence to collect

Code commits, screenshots, working demo, README, report, reflection notes.

Thank you